```
# Name : Prathamesh Lavange
# div : B
# Roll No :31
# PRN : 72332154L
```

```
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime
from getpass import getpass
from sklearn.linear_model import LinearRegression
```

```
# Get a key from https://openweathermap.org/api
API_KEY = getpass("Enter your OpenWeatherMap API key: ")

city = "Pune"
country = "IN"
url = "https://api.openweathermap.org/data/2.5/weather"

params = {
    "q": f"{city},{country}",
    "appid": API_KEY,
    "units": "metric"
}
```

```
Enter your OpenWeatherMap API key: ··········
```

```
response = requests.get(url, params=params, timeout=30)
print("Status Code:", response.status_code)

if response.status_code != 200:
    print("Could not retrieve weather data.")
    print("API response:", response.text)
    raise RuntimeError(
        "Check your API key, internet connection, and OpenWeatherMap API access, then rerun this cell."
    )

print("Data retrieved successfully!")
weather_data = response.json()
weather_data
```

```
Status Code: 200
Data retrieved successfully!
{'coord': {'lon': 73.8553, 'lat': 18.5196},
 'weather': [{'id': 800,
   'main': 'Clear',
   'description': 'clear sky',
   'icon': '01d'}],
 'base': 'stations',
 'main': {'temp': 34.11,
  'feels_like': 31.66,
  'temp_min': 34.11,
  'temp_max': 34.11,
  'pressure': 1012,
  'humidity': 14,
  'sea_level': 1012,
  'grnd_level': 940},
 'visibility': 10000,
 'wind': {'speed': 3.39, 'deg': 73, 'gust': 4.54},
 'clouds': {'all': 5},
 'dt': 1791801019,
 'sys': {'type': 2,
  'id': 2104163,
  'country': 'IN',
  'sunrise': 1791766637,
  'sunset': 1791809083},
 'timezone': 19800,
 'id': 1259229,
 'name': 'Pune',
 'cod': 200}
```

```
weather_record = {
    "City": weather_data["name"],
    "Temperature": weather_data["main"]["temp"],
    "Feels_Like": weather_data["main"]["feels_like"],
    "Min_Temperature": weather_data["main"]["temp_min"],
```

```
    "Max_Temperature": weather_data["main"]["temp_max"],
    "Pressure": weather_data["main"]["pressure"],
    "Humidity": weather_data["main"]["humidity"],
    "Wind_Speed": weather_data.get("wind", {}).get("speed", np.nan),
    "Weather": weather_data["weather"][0]["description"]
}

weather_record
```

```
{'City': 'Pune',
 'Temperature': 34.11,
 'Feels_Like': 31.66,
 'Min_Temperature': 34.11,
 'Max_Temperature': 34.11,
 'Pressure': 1012,
 'Humidity': 14,
 'Wind_Speed': 3.39,
 'Weather': 'clear sky'}
```

```
df = pd.DataFrame([weather_record])
display(df)
```

	City	Temperature	Feels_Like	Min_Temperature	Max_Temperature	Pressure	Humidity	Wind_Speed	Weather
0	Pune	34.11	31.66	34.11	34.11	1012	14	3.39	clear sky

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1 entries, 0 to 0
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   City             1 non-null      object
 1   Temperature      1 non-null      float64
 2   Feels_Like       1 non-null      float64
 3   Min_Temperature  1 non-null      float64
 4   Max_Temperature  1 non-null      float64
 5   Pressure         1 non-null      int64
 6   Humidity         1 non-null      int64
 7   Wind_Speed       1 non-null      float64
 8   Weather          1 non-null      object
dtypes: float64(5), int64(2), object(2)
memory usage: 204.0+ bytes
```

```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
City               0
Temperature        0
Feels_Like         0
Min_Temperature    0
Max_Temperature    0
Pressure           0
Humidity           0
Wind_Speed         0
Weather            0
dtype: int64
```

```
print("Data types:")
print(df.dtypes)
```

```
Data types:
City                object
Temperature        float64
Feels_Like         float64
Min_Temperature    float64
Max_Temperature    float64
Pressure             int64
Humidity             int64
Wind_Speed         float64
Weather             object
dtype: object
```

```
def temperature_category(temp):
    if pd.isna(temp):
        return "Unknown"
```

```
    elif temp < 15:
        return "Cold"
    elif temp < 25:
        return "Moderate"
    elif temp < 35:
        return "Warm"
    else:
        return "Hot"

df["Temperature_Category"] = df["Temperature"].apply(temperature_category)
display(df)
```

	City	Temperature	Feels_Like	Min_Temperature	Max_Temperature	Pressure	Humidity	Wind_Speed	Weather	Temperature_Cat
0	Pune	34.11	31.66	34.11	34.11	1012	14	3.39	clear sky	

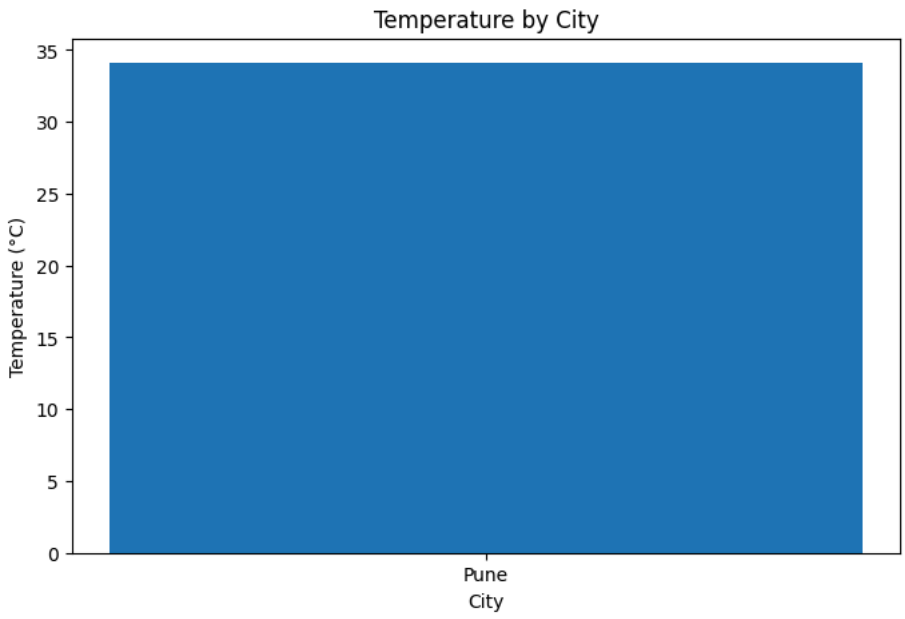
```
df["Date_Time"] = datetime.now()
display(df)
```

	City	Temperature	Feels_Like	Min_Temperature	Max_Temperature	Pressure	Humidity	Wind_Speed	Weather	Temperature_Cat
0	Pune	34.11	31.66	34.11	34.11	1012	14	3.39	clear sky	

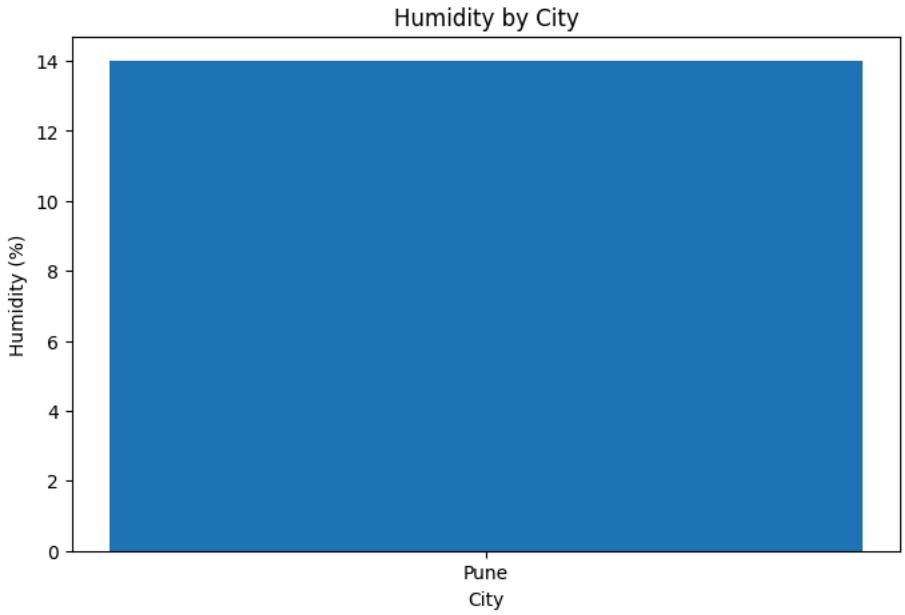
```
print("Temperature:", df["Temperature"].mean())
print("Humidity:", df["Humidity"].mean())
print("Pressure:", df["Pressure"].mean())
print("Wind Speed:", df["Wind_Speed"].mean())
```

```
Temperature: 34.11
Humidity: 14.0
Pressure: 1012.0
Wind Speed: 3.39
```

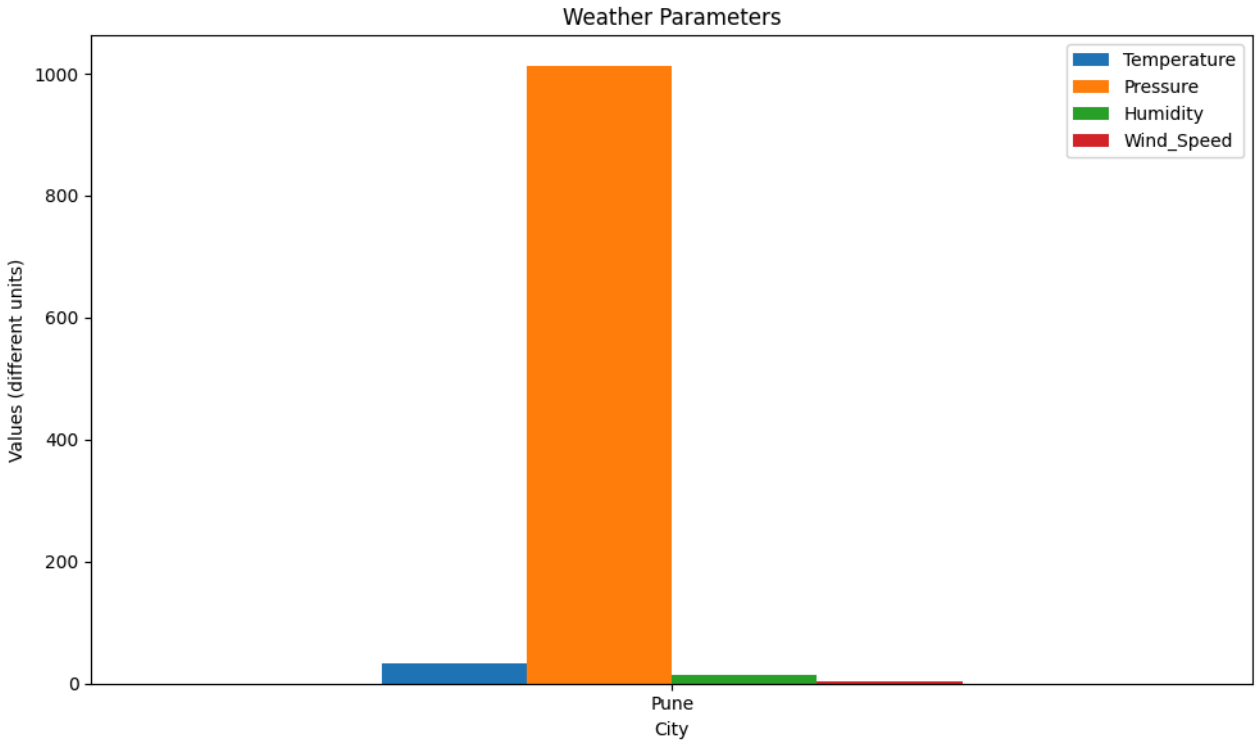
```
plt.figure(figsize=(8, 5))
plt.bar(df["City"], df["Temperature"])
plt.title("Temperature by City")
plt.xlabel("City")
plt.ylabel("Temperature (°C)")
plt.show()
```




```
plt.figure(figsize=(8, 5))
plt.bar(df["City"], df["Humidity"])
plt.title("Humidity by City")
plt.xlabel("City")
plt.ylabel("Humidity (%)")
plt.show()
```



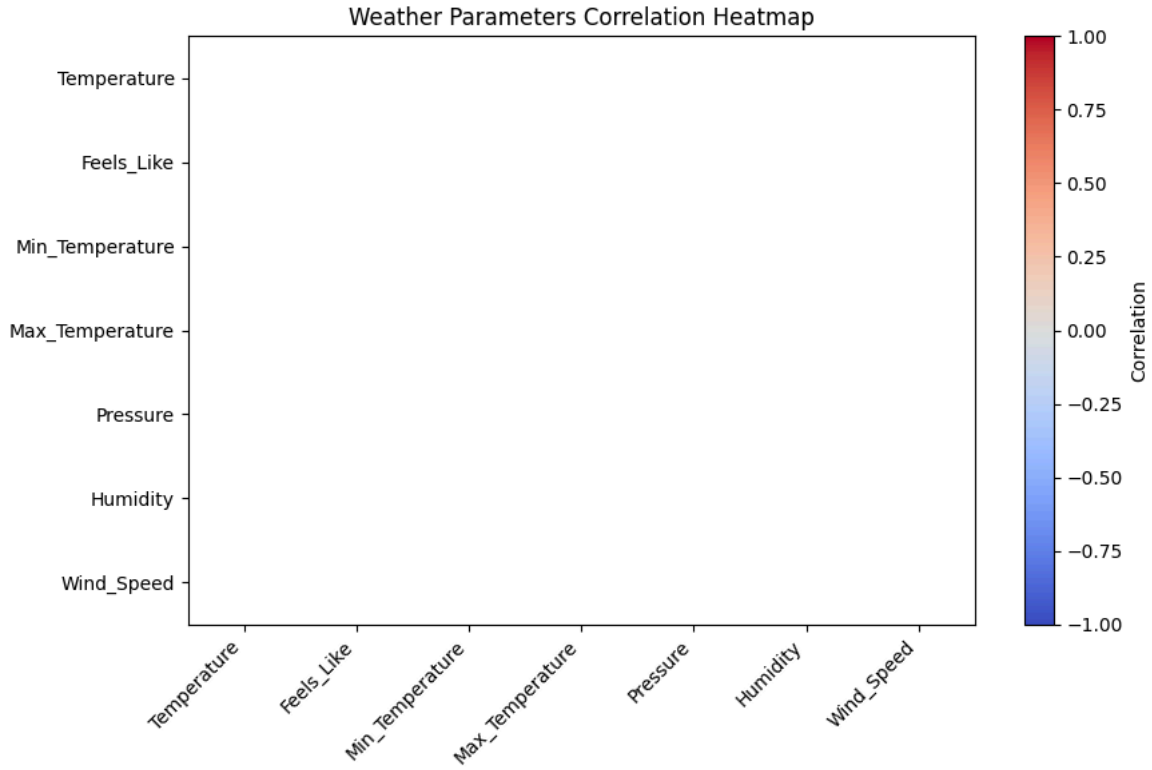

```
weather_parameters = ["Temperature", "Pressure", "Humidity", "Wind_Speed"]
df.set_index("City")[weather_parameters].plot(
    kind="bar", figsize=(10, 6)
)
plt.title("Weather Parameters")
plt.ylabel("Values (different units)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```




```
numeric_columns = [
    "Temperature",
    "Feels_Like",
    "Min_Temperature",
    "Max_Temperature",
    "Pressure",
    "Humidity",
    "Wind_Speed"
]
correlation = df[numeric_columns].corr()
display(correlation)
```

```
plt.figure(figsize=(9, 6))
plt.imshow(correlation, cmap="coolwarm", vmin=-1, vmax=1, aspect="auto")
plt.colorbar(label="Correlation")
plt.xticks(range(len(numeric_columns)), numeric_columns, rotation=45, ha="right")
plt.yticks(range(len(numeric_columns)), numeric_columns)
plt.title("Weather Parameters Correlation Heatmap")
plt.tight_layout()
plt.show()
```

	Temperature	Feels_Like	Min_Temperature	Max_Temperature	Pressure	Humidity	Wind_Speed
Temperature	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Feels_Like	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Min_Temperature	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Max_Temperature	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Pressure	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Humidity	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Wind_Speed	NaN	NaN	NaN	NaN	NaN	NaN	NaN



Next steps: Generate code with correlation | New interactive sheet

```
# Note: one API observation is not enough for a meaningful regression model.
# This follows the PDF's basic example, but the prediction is only illustrative.
X = df[["Temperature"]]
y = df["Feels_Like"]

model = LinearRegression()
model.fit(X, y)

df["Predicted_Feels_Like"] = model.predict(X)
display(df)
```

	City	Temperature	Feels_Like	Min_Temperature	Max_Temperature	Pressure	Humidity	Wind_Speed	Weather	Temperature_Cat
0	Pune	34.11	31.66	34.11	34.11	1012	14	3.39	clear sky	

```
df.to_csv("pune_weather_data.csv", index=False)
print("Saved file: pune_weather_data.csv")

# In Google Colab, run this next cell to download the CSV:
# from google.colab import files
```

```
# from google.colab import files
# files.download("pune_weather_data.csv")
```

```
Saved file: pune_weather_data.csv
```